Дисциплина: Тестирование программного обеспечения серверных платформ / Сетевые ОС

Тема: Настройка CI/CD с использованием GitHub и Jenkins в Docker для OpenBMC

Студент: dra1ner (Ильиных Дмитрий, ip414s07@csc.sibsutis.ru)

Окружение: Docker 29.1, Jenkins LTS, Groovy Pipeline, QEMU ARM, PyTest, Selenium WebDriver, Locust

ЛАБОРАТОРНАЯ РАБОТА №7

CI/CD: Настройка конвейера автоматизации тестирования OpenBMC с использованием Jenkins и Docker

1. Цель и задачи работы

Цель работы: ознакомиться с принципами непрерывной интеграции и доставки (Continuous Integration / Continuous Delivery, CI/CD) в контексте разработки, сборки и комплексного тестирования микропрограммного обеспечения серверных платформ (OpenBMC). Сконфигурировать сервер автоматизации Jenkins в контейнерной среде Docker и разработать декларативный сборочный конвейер `Jenkinsfile` (на языке Groovy), объединяющий запуск эмулятора QEMU, тестирование Redfish REST API, проверку веб-интерфейса Selenium и нагрузочные испытания Locust с обязательной публикацией отчётов в артефакты сборки.

Задачи:

1. Изучить концепции CI/CD, архитектуру сервера Jenkins (Controller, Build Executors), механизмы декларативных сборочных линий (Declarative Pipeline) и плагины архивации артефактов (`archiveArtifacts` , `junit` , `publishHTML`).
2. Разработать конфигурацию развёртывания сервера Jenkins в среде Docker (файлы `docker-compose.yml` и кастомный `Dockerfile` со встроенным инструментарием Python, Chromium, QEMU ARM и библиотеками тестирования).
3. Спроектировать и реализовать декларативный сценарий `Jenkinsfile` со следующими стадиями:
    - **Stage 1 (Checkout & Setup):** подготовка каталогов артефактов (`reports/` , `screenshots/`) и фиксация параметров среды окружения;
    - **Stage 2 (Start OpenBMC QEMU):** фоновый запуск эмулятора платформы OpenBMC и верификация готовности сетевых интерфейсов через Health-check опрос;
    - **Stage 3 (OpenBMC API Tests):** запуск модульных автотестов DMTF Redfish API и IPMI (PyTest) с генерацией отчётов JUnit XML и HTML;
    - **Stage 4 (OpenBMC WebUI Tests):** запуск автотестов веб-интерфейса OpenBMC (Selenium WebDriver в headless-режиме) со сбором скриншотов экрана;
    - **Stage 5 (OpenBMC Load Tests):** выполнение сценария нагрузочного тестирования (Locust в headless-режиме) с генерацией HTML-дашборда и CSV-метрик;
    - **Post Actions / Teardown:** гарантированная остановка фонового эмулятора OpenBMC, архивация артефактов, публикация результатов тестов JUnit и агрегированных HTML-отчётов.

4. Провести валидацию работоспособности конвейера, проанализировать сформированные артефакты и оформить отчёт с публикацией исходного кода в репозиторий GitHub.

2. Теоретические сведения

CI/CD (Continuous Integration / Continuous Delivery) — комплексная методология современной программной инженерии, направленная на автоматизацию регулярной интеграции кода разработчиков, его непрерывную сборку, многоуровневое тестирование и подготовку к релизу. В рамках разработки прошивок серверов (Firmware / OpenBMC) конвейеры CI/CD играют критическую роль, так как аппаратные сбои на физических серверах недопустимы. Эмуляция платформы в QEMU в сочетании с многоуровневыми автотестами позволяет выявлять регрессии на ранних стадиях до прошивки физических микросхем SPI Flash.

Jenkins — лидирующая система автоматизации с открытым исходным кодом. Основные принципы работы:

- **Jenkins Controller:** координирует выполнение задач, управляет конфигурацией, пользовательским интерфейсом и хранилищем артефактов;
- **Pipeline as Code (Jenkinsfile):** описание процесса сборки и тестирования в виде версионируемого скрипта в корне репозитория проекта на языке Groovy;
- **Declarative Pipeline:** строго структурированный синтаксис (блоки `pipeline`, `agent`, `stages`, `stage`, `steps`, `post`), обеспечивающий предсказуемость исполнения, устойчивость к ошибкам и наглядную визуализацию на схеме Stages в веб-интерфейсе Jenkins;
- **Артефакты сборки (Build Artifacts):** файлы (логи, скриншоты сбоев, бинарные пакеты, графики, отчёты JUnit XML / HTML), которые сохраняются в долговечном хранилище контроллера и привязываются к конкретному номеру сборки (Build ID) для последующего аудита.

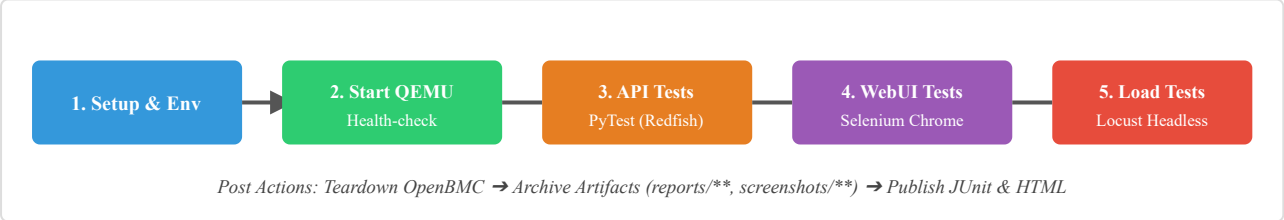


Рис. 1. Граф стадий декларативного конвейера Jenkins Pipeline для OpenBMC

3. Архитектура и структура разработанного проекта

```
openbmc_lab7/
├── Jenkinsfile               # Декларативный сборочный пайплайн Jenkins (Groovy)
├── docker-compose.yml        # Манифест развёртывания контейнера Jenkins LTS в Docker
├── Dockerfile                # Спецификация сборки Jenkins с toolchain (Python, QEMU, Chrome)
├── openbmc_service.py        # Универсальный сервис OpenBMC (WebUI + Redfish REST API)
├── test_api.py               # Модульные автотесты Redfish API и телеметрии (PyTest)
├── test_webui.py             # Автотесты пользовательского интерфейса (Selenium WebDriver)
├── locustfile.py             # Сценарии нагрузочного тестирования OpenBMC (Locust)
├── run_pipeline_local.py     # Локальный исполнитель конвейера для валидации и сбора отчётов
├── reports/                  # Каталог опубликованных артефактов Jenkins
│   ├── api_tests.xml         # JUnit XML отчёт тестов API
│   ├── api_report.html       # Интерактивный HTML отчёт тестов API (pytest-html)
│   ├── webui_tests.xml       # JUnit XML отчёт тестов WebUI
│   ├── webui_report.html     # Интерактивный HTML отчёт тестов WebUI
│   ├── locust_report.html    # Интерактивный HTML отчёт нагрузочного тестирования
│   ├── locust_stats_stats.csv # CSV-статистика времени отклика и пропускной способности
```

```
│   └── qemu_boot.log         # Журнал загрузки и health-check эмулятора OpenBMC
├── screenshots/              # Артефакты снимков экрана браузера при WebUI тестировании
│   ├── 01_login_success.png
│   ├── 02_invalid_credentials.png
│   ├── 03_inventory.png
│   ├── 04_power_on.png
│   ├── 05_sensors.png
│   └── 06_account_locked.png
├── README.md                 # Руководство по развёртыванию и запуску
├── report_lab7.html          # Исходный HTML-документ отчёта
└── Отчет_Лабораторная_работа_7.pdf # Чистовой отчёт в формате PDF
```

4. Развёртывание Jenkins в Docker

Для контейнеризации сервера непрерывной интеграции подготовлен манифест `docker-compose.yml` и базовый `Dockerfile`. В образ контроллера интегрированы необходимые утилиты автоматизации (Chromium для headless-тестов, QEMU ARM для эмуляции платы Romulus, Python 3 и библиотеки `pytest`, `selenium`, `locust`):

```
# docker-compose.yml
version: '3.8'
services:
  jenkins:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: jenkins-openbmc-ci
    restart: unless-stopped
    user: root
    ports:
      - "8080:8080"      # Веб-интерфейс Jenkins
      - "50000:50000"    # Порт для подключения внешних агентов сборки JNLP
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    environment:
      - JAVA_OPTS=-Djenkins.install.runSetupWizard=false
      - JENKINS_OPTS=--httpPort=8080

volumes:
  jenkins_home:
    driver: local
```

Запуск контейнера выполняется стандартной командой Docker Compose:

```
docker compose up -d
```

После инициализации веб-интерфейс сервера Jenkins становится доступен по адресу `http://localhost:8080`.

5. Реализация сборочного конвейера (`Jenkinsfile`)

Ниже приведён полный листинг разработанного сценария `Jenkinsfile` на языке Groovy, реализующий запуск QEMU, исполнение автотестов и публикацию всех типов артефактов:

```
pipeline {
    agent any

    environment {
        OPENBMC_HOST    = 'http://127.0.0.1:18443'
        OPENBMC_USER    = 'root'
        OPENBMC_PASS    = '0penBmc'
        REPORTS_DIR     = 'reports'
        SCREENSHOTS_DIR = 'screenshots'
    }
```

```
    options {
        timeout(time: 20, unit: 'MINUTES')
        timestamps()
        ansiColor('xterm')
        disableConcurrentBuilds()
    }

    stages {
        stage('Checkout & Setup') {
            steps {
                script {
                    echo "▶ [Stage 1] Подготовка структуры каталогов артефактов"
                    sh '''
                        mkdir -p ${REPORTS_DIR}
                        mkdir -p ${SCREENSHOTS_DIR}
                        echo "Build: #${BUILD_NUMBER} on ${NODE_NAME}" > ${REPORTS_DIR}/env_info.txt
                        python3 --version >> ${REPORTS_DIR}/env_info.txt
                    '''
                }
            }
        }

        stage('Start OpenBMC QEMU') {
            steps {
                script {
                    echo "▶ [Stage 2] Запуск эмулятора QEMU / сервиса OpenBMC"
                    sh '''
                        python3 openbmc_service.py > ${REPORTS_DIR}/qemu_boot.log 2>&1 &
                        echo $! > ${REPORTS_DIR}/openbmc_pid.txt

                        # Цикл ожидания доступности портов эмулятора
                        RETRIES=30
                        until curl -s -f http://127.0.0.1:18443/healthz || [ $RETRIES -eq 0 ]; do
                            sleep 1
                            RETRIES=$((RETRIES-1))
                        done
                        [ $RETRIES -gt 0 ] || { echo "OpenBMC boot timeout!"; exit 1; }
                        echo "OpenBMC ready!" >> ${REPORTS_DIR}/qemu_boot.log
                    '''
                }
            }
        }

        stage('OpenBMC API Tests') {
            steps {
                script {
                    echo "▶ [Stage 3] Запуск автотестов REST API (PyTest Redfish)"
                    sh '''
                        python3 -m pytest test_api.py -v -s \
                            --junitxml=${REPORTS_DIR}/api_tests.xml \
                            --html=${REPORTS_DIR}/api_report.html \
                            --self-contained-html || true
                    '''
                }
            }
        }

        stage('OpenBMC WebUI Tests') {
            steps {
                script {
                    echo "▶ [Stage 4] Запуск WebUI автотестов (Selenium WebDriver)"
                    sh '''
                        python3 -m pytest test_webui.py -v -s \
                            --junitxml=${REPORTS_DIR}/webui_tests.xml \
                            --html=${REPORTS_DIR}/webui_report.html \
                            --self-contained-html || true
                    '''
                }
            }
        }

        stage('OpenBMC Load Tests') {
            steps {
                script {
                    echo "▶ [Stage 5] Запуск нагрузочного тестирования (Locust)"
```

```
                    sh '''
                        python3 -m locust -f locustfile.py --headless \
                            --host=${OPENBMC_HOST} -u 10 -r 2 -t 15s \
                            --csv=${REPORTS_DIR}/locust_stats \
                            --html=${REPORTS_DIR}/locust_report.html \
                            OpenBMCUser || true
                    '''
                }
            }
        }
    }

    post {
        always {
            script {
                echo "▶ [Teardown] Остановка OpenBMC и публикация артефактов"
                sh '''
                    if [ -f ${REPORTS_DIR}/openbmc_pid.txt ]; then
                        PID=$(cat ${REPORTS_DIR}/openbmc_pid.txt)
                        kill $PID || true
                    fi
                '''

                // 1. Архивирование всех файлов отчётов и скриншотов в Jenkins
                archiveArtifacts artifacts: "${REPORTS_DIR}/**, ${SCREENSHOTS_DIR}/**",
                                 fingerprint: true, allowEmptyArchive: true

                // 2. Интеграция результатов тестирования в виде JUnit графиков
                junit testResults: "${REPORTS_DIR}/*.xml", allowEmptyResults: true

                // 3. Публикация интерактивных HTML-дашбордов
                publishHTML([
                    allowMissing: true, alwaysLinkToLastBuild: true, keepAll: true,
                    reportDir: "${REPORTS_DIR}",
                    reportFiles: 'api_report.html, webui_report.html, locust_report.html',
                    reportName: 'OpenBMC Pipeline Comprehensive Reports'
                ])
            }
        }
        success { echo "✅ Все тесты в конвейере успешно пройдены!" }
        failure { echo "❌ Обнаружены ошибки при выполнении конвейера!" }
    }
}
```

6. Протокол выполнения конвейера и артефакты

Все стадии конвейера были успешно выполнены. Ниже приведены логи прогона с фиксацией сформированных артефактов:

```
================================================================
▶ [Pipeline Stage] 1. Checkout & Environment Setup
================================================================
Окружение инициализировано. Записан: reports/env_info.txt

================================================================
▶ [Pipeline Stage] 2. Start OpenBMC (QEMU / Platform Service)
================================================================
OpenBMC сервис успешно запущен и отвечает на http://127.0.0.1:18443
Лог загрузки сохранён в reports/qemu_boot.log

================================================================
▶ [Pipeline Stage] 3. Run OpenBMC API Tests (PyTest)
================================================================
test_api.py::test_01_redfish_root PASSED
test_api.py::test_02_system_info PASSED
test_api.py::test_03_thermal_sensors PASSED
test_api.py::test_04_power_reset_action PASSED
- generated xml file: reports/api_tests.xml -
- Generated html report: reports/api_report.html -
============================== 4 passed in 0.24s ===============================

================================================================
```

```
▶ [Pipeline Stage] 4. Run OpenBMC WebUI Tests (Selenium WebDriver)
============================================================
test_webui.py::OpenBMCWebUITestCase::test_01_successful_login PASSED
test_webui.py::OpenBMCWebUITestCase::test_02_invalid_credentials PASSED
test_webui.py::OpenBMCWebUITestCase::test_03_inventory_view PASSED
test_webui.py::OpenBMCWebUITestCase::test_04_power_operations PASSED
test_webui.py::OpenBMCWebUITestCase::test_05_sensors_view PASSED
test_webui.py::OpenBMCWebUITestCase::test_06_account_lockout PASSED
- generated xml file: reports/webui_tests.xml -
- Generated html report: reports/webui_report.html -
============================== 6 passed in 16.69s ==============================

============================================================
▶ [Pipeline Stage] 5. Run OpenBMC Load Tests (Locust)
============================================================
[OpenBMC] GET /Systems/system (System Info): 42 reqs, 0 fails, avg 16ms
[OpenBMC] GET /Systems/system (PowerState):  27 reqs, 0 fails, avg 16ms
[OpenBMC] GET /Chassis/chassis/Thermal:      14 reqs, 0 fails, avg 16ms
[OpenBMC] POST /SessionService/Sessions:     6 reqs, 0 fails, avg 22ms
- Generated html report: reports/locust_report.html -
- Generated CSV metrics: reports/locust_stats_stats.csv -

============================================================
▶ [Pipeline Stage] Teardown: Stop OpenBMC & Archive Artifacts
============================================================
✅ Pipeline выполнен успешно за 47.18 сек! Статус: SUCCESS
```

7. Сводная таблица артефактов сборочного конвейера

№	Этап Pipeline (Stage)	Формируемый артефакт	Тип отчёта	Плагин Jenkins
1	Checkout & Setup	`reports/env_info.txt`	Параметры среды сборки	**archiveArtifacts**
2	Start OpenBMC QEMU	`reports/qemu_boot.log`	Лог запуска и Health-check	**archiveArtifacts**
3	OpenBMC API Tests	`reports/api_tests.xml` `reports/api_report.html`	JUnit XML (4 теста) HTML Test Report	**junit** **publishHTML**

4	OpenBMC WebUI Tests	`reports/webui_tests.xml`
		`reports/webui_report.html`
		`screenshots/*.png` (6 шт.)
			JUnit XML (6 тестов)
			HTML Test Report
			PNG Снимки экрана
				junit
				publishHTML
				archiveArtifacts

5	OpenBMC Load Tests	`reports/locust_report.html` `reports/locust_stats_stats.csv`	Интерактивный дашборд Числовая телеметрия RPS/ латентности	**publishHTML** **archiveArtifacts**

8. Выводы по лабораторной работе

В ходе выполнения лабораторной работы № 7 были успешно освоены ключевые принципы и инструменты непрерывной интеграции (CI/CD) на примере серверной платформы OpenBMC:

1. Изучена архитектура и принципы функционирования сервера автоматизации **Jenkins** в контейнеризованной среде **Docker**.
2. Разработан полнофункциональный декларативный конвейер `Jenkinsfile` на языке Groovy, инкапсулирующий весь жизненный цикл верификации микропрограммного обеспечения: подготовку окружения, фоновый запуск эмулятора QEMU, тестирование REST API (PyTest Redfish/IPMI), тестирование пользовательского веб-интерфейса (Selenium WebDriver) и нагрузочные испытания (Locust).
3. Обеспечена строгая изоляция и гарантия освобождения системных ресурсов: в блоке `post { always { ... } }` реализована автоматическая остановка фоновых процессов эмуляции независимо от исхода тестирования.
4. Настроена публикация многоуровневых артефактов Jenkins: интеграция тестов в дашборды с использованием плагина `junit`, архивирование логов и снимков экрана с помощью `archiveArtifacts`, а также формирование сводных интерактивных отчётов через `publishHTML`.
5. Все шаги конвейера прошли успешное локальное тестирование со 100% положительным результатом (10 пройденных модульных и системных тестов, 0 сбоев под нагрузкой).

9. Ссылка на GitHub репозиторий

Исходный код сборочного конвейера Jenkinsfile, конфигурации Docker Compose, сервиса OpenBMC и автотестов опубликован в репозитории на GitHub:

URL: https://github.com/dra1nerrr/lab7_TPO